

Interview Prep: Tokens, Embeddings, Attention & GPT

15 common conceptual questions, with concise interview-ready answers

Tokenization & Embeddings

Q: What is tokenization, and why do modern LLMs use subword tokenization instead of word-level tokenization?

A: Tokenization splits raw text into discrete units a model can process. Subword tokenization (e.g. BPE) splits text into pieces smaller than whole words — common words stay whole, rare words break into known sub-pieces. This keeps vocabulary size manageable (tens of thousands of tokens instead of an unbounded number of words) while gracefully handling any word, even ones never seen during training, instead of falling back to a generic <UNK> token.

Q: What's the actual difference between a learned embedding and something like TF-IDF or one-hot encoding?

A: TF-IDF and one-hot encoding are fixed, hand-computed representations based on frequency statistics or identity — they carry no notion of meaning or relationship between words. A learned embedding is a dense vector that starts random and is shaped during training, just like any other model weight. Words used in similar contexts tend to end up with similar vectors purely as a side effect of the model minimizing its loss — nobody hand-labels these relationships.

Q: Why can't you just feed raw token IDs directly into a neural network?

A: Token IDs are arbitrary integers assigned during vocabulary construction — there's no meaningful relationship between, say, token 482 and token 483 just because they're numerically close. Feeding raw IDs in would imply a false ordering/relationship. An embedding layer maps each ID to a learned vector instead, so the network operates on a representation that can actually capture similarity.

Self-Attention & Transformers

Q: Explain self-attention in your own words. What do Query, Key, and Value represent?

A: Self-attention lets each token dynamically decide how much to weight every other token in the sequence, rather than treating them all equally (as mean pooling would) or only looking at a fixed local window (as convolution would). Query represents what a token is 'looking for'; Key represents what each token 'advertises' it contains; Value is the actual content passed along once a token is deemed relevant. Q, K, and V are all learned linear projections of the same input embeddings.

Q: Walk through the scaled dot-product attention formula. Why do we divide by $\sqrt{d_k}$?

A: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$. QK^T computes a relevance score between every pair of tokens; softmax turns each row into weights summing to 1; multiplying by V produces a weighted combination of token content. Dividing by $\sqrt{d_k}$ prevents dot products from growing too large as dimensionality increases, which would otherwise push softmax into a saturated region with extremely small gradients, slowing or stalling learning.

Q: What is positional encoding, and why does a Transformer need it?

A: Self-attention's computation is purely content-based — it has no inherent notion of token order; shuffling the input produces the same set of outputs, just reordered. Positional encoding adds a fixed, unique, position-dependent vector to each token's embedding before attention, so the model has access to ordering information. The original Transformer paper uses sine/cosine functions at varying frequencies to give every position a distinct fingerprint.

Q: What's the difference between single-head and multi-head attention?

A: A single attention computation can only learn one way of relating tokens. Multi-head attention runs several independent attention computations in parallel (each with its own Q/K/V weight matrices), letting different heads potentially specialize in different kinds of relationships — not hand-designed, they emerge from training. The outputs are concatenated and passed through one more linear layer to return to the original dimensionality.

Q: What is causal masking, and why does a model like GPT need it?

A: Causal masking restricts each token to only attend to itself and earlier positions, never future ones. This is required for autoregressive generation — at generation time future tokens genuinely don't exist yet, and training must match that constraint or the model could 'cheat' by seeing the answer. Implementation: set future positions to -infinity in the attention scores before softmax, so $\text{softmax}(-\infty)=0$ and those positions get exactly zero weight, while rows still correctly sum to 1.

Q: What role do residual connections and layer normalization play in a Transformer block?

A: Residual connections add a sublayer's input back to its output ($x + \text{sublayer}(x)$), which helps gradients flow through many stacked layers without vanishing — important given how deep modern Transformers are. Layer normalization

normalizes activations per-token across the feature dimension, stabilizing training. Neither is unique to Transformers, but both are essential to making very deep stacks of them actually trainable.

Q: What's the difference between an encoder and a decoder in the Transformer architecture, and why is GPT decoder-only?

A: The original Transformer (for translation) has two halves: an encoder that reads the full input with no masking, and a decoder that generates output causally, also attending back to the encoder's output. GPT keeps only the decoder half — causal self-attention, no separate encoder — because its only job is generating text left-to-right one token at a time, not translating between two distinct sequences.

Q: Why did attention-based Transformers largely replace RNNs/LSTMs for sequence modeling?

A: RNNs process sequences one step at a time, maintaining a hidden state carried forward — this makes them inherently sequential (hard to parallelize) and prone to vanishing gradients over long sequences, since gradients must flow backward through every time step. Self-attention lets every token directly access every other token in a single computation, regardless of distance, sidestepping the vanishing-gradient problem and enabling much better parallelization during training.

GPT & LLMs

Q: What is GPT actually trained to do? What's the training objective?

A: Next-token prediction: given a sequence of tokens, predict the next one. Every position in a training sequence provides its own prediction target for free — the target at each position is simply 'the next token,' with no manual labeling required. This is why pretraining can use enormous amounts of raw text with no annotation cost, unlike supervised learning.

Q: What's the difference between pretraining and fine-tuning?

A: Pretraining trains a model from scratch (or near-scratch) on a large, generic corpus using a general objective like next-token prediction, producing broad language competence. Fine-tuning takes that pretrained model and continues training it, usually with a smaller, task-specific dataset, to specialize it for a particular use case (e.g., sentiment classification, instruction-following) without starting over.

Q: Explain temperature and top-k sampling in text generation. What tradeoff do they control?

A: Both control the randomness of picking the next token from the model's predicted probability distribution. Temperature scales the logits before softmax: values below 1 sharpen the distribution (more confident, more repetitive), values above 1 flatten it (more diverse, more mistakes). Top-k restricts sampling to only the k most likely tokens, discarding the long tail entirely, which prevents the model from ever picking a wildly unlikely token while still allowing some randomness among reasonable choices. Both trade off coherence against diversity.

Q: What is a context window, and why is it limited rather than unlimited?

A: The context window is the maximum sequence length a model can process in a single forward pass. It's limited because self-attention's computation and memory cost scales quadratically with sequence length (every token attends to every other token) — doubling the context window roughly quadruples the compute cost of attention, which is why extending context windows is a genuine engineering challenge, not just a configuration setting.